Automated Citation Verification System

Operations Guide: Pipeline Execution, Testing & Real-World Dataset Migration

Target Audience: Data Engineering, ML, QA & Verification Teams	**Corpus Domains:** Legal Statutes/Judgments & Scientific Papers
Pipeline Version: 1.0.0-Stage11 (End-to-End Audited)	**Test Status:** 243/243 Automated Unit & Integration Tests Passing

Executive Summary & Guide Purpose

This operations manual specifies instructions for setting up the environment, running the 243-test automated test suite, and replacing synthetic test data with the production dataset prepared by the data team. The system treats the database as an **evidence store** with complete structural provenance (page coordinates, exact section hierarchies, table schemas, and parent-child clause links) ensuring high-fidelity claim verification.

1. System Architecture & Lifecycle Stages

The verification engine is architected into three tightly integrated, deterministic stages:

Pipeline Phase	Stage Range	Primary Function & Key Technologies
Offline Corpus Building	**Stages 0 - 6**	Document parsing (Docling), structure-aware chunking (legal regex / scientific headings), metadata normalization, domain embeddings (BGE-Large for Legal / SPECTER2 for Scientific), persistent ChromaDB indexing.
Runtime Verification	**Stages 7 - 10C**	Claim/citation detection, bibliography reference resolution, dual-mode retrieval (exact section match + dense vector), source attribution isolation, closed-world LLM semantic verification.
Evaluation & Diagnostics	**Stage 11**	Version-controlled benchmark suites, stage-level error attribution (14 failure categories), automated verdict confusion matrices, and audit report generation.

Core Verification Verdict Taxonomy: Every query produces one of five auditable verdicts: **supported** (explicitly proven by evidence), **unsupported** (evidence mentions context but fails to prove claim), **contradicted** (evidence refutes claim), **partial** (proves subset of multi-part claim), or **not_found** (cited statute/reference does not exist).

2. Environment Setup & Running Existing Code

2.1 Prerequisites & Virtual Environment Setup

The project requires Python 3.11 or 3.12 with PyTorch and Docling. Clone the repository and configure the environment:

Terminal Commands -- Environment Setup

```
# 1. Clone repository
git clone https://github.com/dharani20dharan/Citation-Verification.git
cd Citation-Verification

# 2. Create and activate virtual environment
# Windows PowerShell:
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# Linux / macOS:
python3 -m venv .venv
source .venv/bin/activate

# 3. Install dependencies
pip install -r requirements.txt
```

2.2 Environment Configuration (.env)

Copy `.env.example` to `.env`. API keys are only necessary when running Stage 10C LLM verification. The corpus builder and evaluation benchmarks operate fully offline and deterministically.

.env Configuration Template

```
CHROMA_PERSIST_DIR=./vector_db/chroma
SCIENTIFIC_EMBEDDING_MODEL=allenai/specter2_base
LEGAL_EMBEDDING_MODEL=BAAI/bge-large-en-v1.5
EMBEDDING_DEVICE=auto   # Options: auto, cuda, cpu
OPENAI_API_KEY=your_key_here          # Optional: Stage 10C LLM semantic verifier
GEMINI_API_KEY=your_key_here          # Optional: Stage 10C LLM semantic verifier
```

2.3 Running the Automated Test Suite

Ensure all baseline tests pass before initiating dataset migration. The project includes 243 automated tests:

Running Pytest

```
# Run full test suite through virtual environment:
.\.venv\Scripts\pytest -v

# Run domain-specific unit test suites:
.\.venv\Scripts\pytest tests/test_legal_chunker.py
.\.venv\Scripts\pytest tests/test_scientific_chunker.py
.\.venv\Scripts\pytest tests/test_references.py
.\.venv\Scripts\pytest tests/test_retrieval.py
.\.venv\Scripts\pytest tests/test_integration_pipeline.py
```

> **Test Suite Determinism Guarantee**
>
> The test suite verifies domain isolation (legal and scientific corpora never mix), exact reference resolution (e.g., Section 3(d) does not match Section 3(e) or 3(d)(i)), parent-context expansion, and the 14-category error taxonomy. All 243 tests must show green before proceeding with real dataset integration.

3. Replacing Synthetic Data with Real-World Dataset

The repository contains synthetic fixtures in `data/` for validation. Follow the mapping below to replace these fixtures with authentic production assets prepared by the data team:

3.1 Data Directory Mapping

Directory Path	Current Synthetic Data	Real-World Replacement Action
`data/raw/legal/`	Sample fixture PDFs	Deposit all authentic Legal Judgments & Acts (PDF / DOCX).
`data/raw/scientific/`	Sample publication fixtures	Deposit all authentic scientific research papers (PDF / DOCX).
`data/evaluation/`	17-case synthetic `benchmark.jsonl`	Replace with real-world curated ground-truth benchmark suite.
`vector_db/chroma/`	Synthetic indexed embeddings	**Delete contents completely** before re-indexing real dataset.
`data/parsed/ & data/chunks/`	Intermediary stage fixtures	Will be re-populated automatically upon running the pipeline.

3.2 Partner Dataset Integration Modalities

Depending on what format the partner team provides, follow the matching procedure:

- **Option A: Raw Documents (PDF / DOCX):** Deposit files into `data/raw/legal/` and `data/raw/scientific/`. Run the end-to-end ingestion pipeline described in Section 4.
- **Option B: Pre-Chunked JSONL Evidence:** If the team has already segmented the documents, files must conform to `EvidenceChunk` schema. Save under `data/chunks/legal/` and `data/chunks/scientific/`.
- **Option C: Ground-Truth Verification Benchmark:** Replace `data/evaluation/benchmark.jsonl` with their curated evaluation set containing claims, citation strings, and expected verdicts.

Mandatory Schema Compliance for Pre-Chunked Data

Every record in pre-chunked JSONL must validate against **src.models.schemas.EvidenceChunk**:
- **chunk_id** (str, unique) | **document_id** (str) | **domain** ('legal' | 'scientific')
- **title** (str) | **source** (str) | **page** (int ≥ 1) | **section** (str) | **section_id** (str)
- **content_type** ('paragraph' | 'section' | 'clause' | 'table' | 'figure' | 'heading' | 'caption')
- **text** (clean string ≥ 1 char) | **parent_id** (str or null if root)

3.3 Ground-Truth Benchmark Specification

When the team provides the benchmark suite, each line in `data/evaluation/benchmark.jsonl` must adhere to this format:

benchmark.jsonl Record Format

```
{
  "case_id": "bench_legal_real_001",
  "domain": "legal",
  "document_id": "patent_act_1970",
  "claim": "Section 3(d) requires demonstration of enhanced therapeutic efficacy.",
  "reference": "Section 3(d)",
  "normalized_reference": "3(d)",
  "reference_type": "legal_section",
  "expected_reference_status": "exact_match",   # exact_match | not_found | semantic_candidate
  "expected_source_status": "available",        # available | unavailable | unresolved_reference
  "expected_verdict": "supported",              # supported | unsupported | contradicted | partial | not_fo
und
  "expected_conclusive": true,
  "expected_evidence_behavior": {
    "eligible_required": true,
    "required_chunks": ["patent_act_sec_3_d_chunk_01"],
    "prohibited_chunks": ["patent_act_sec_3_heading"]
  }
}
```

4. Step-by-Step Pipeline Re-run with Original Data

Execute the following sequence in terminal to process raw original documents into indexed Chroma evidence stores:

Corpus Building Execution Commands

```
# ------------------------------------------------------------------------
# STEP 0: Reset ChromaDB Indexes and Clean Staging Cache
# ------------------------------------------------------------------------
# Windows PowerShell:
Remove-Item -Recurse -Force vector_db\chroma\*

# ------------------------------------------------------------------------
# STEP 1: Parse Raw PDF/DOCX using Docling Structural Parser
# ------------------------------------------------------------------------
python scripts/parse_document.py --input data/raw/scientific/original_paper.pdf --domain scientific
python scripts/parse_document.py --input data/raw/legal/original_judgment.pdf --domain legal

# ------------------------------------------------------------------------
# STEP 2: Generate Structure-Aware Evidence Chunks
# ------------------------------------------------------------------------
python scripts/build_chunks.py --domain scientific
python scripts/build_chunks.py --domain legal

# ------------------------------------------------------------------------
# STEP 3: Extract Bibliographic References & Section Headings
# ------------------------------------------------------------------------
python scripts/build_references.py --domain scientific

# ------------------------------------------------------------------------
# STEP 4: Compute Domain Embeddings (SPECTER2 for Sci / BGE-Large for Legal)
# ------------------------------------------------------------------------
python scripts/build_embeddings.py --domain scientific
python scripts/build_embeddings.py --domain legal

# ------------------------------------------------------------------------
# STEP 5: Index Chunks into Persistent ChromaDB Collections
# ------------------------------------------------------------------------
python scripts/build_vector_store.py --domain scientific
python scripts/build_vector_store.py --domain legal

# ------------------------------------------------------------------------
# ONE-COMMAND AUTOMATED BUILD (Runs Steps 1 through 5 automatically):
# ------------------------------------------------------------------------
python -m src.pipeline.build_corpus --input data/raw/scientific --domain scientific
python -m src.pipeline.build_corpus --input data/raw/legal --domain legal
```

4.1 Auditing the Rebuilt Evidence Corpus

Validate database health immediately after indexing to verify chunk persistence and metadata retention:

Corpus Validation Execution

```
python scripts/validate_corpus.py

# The validation script performs automated checks:
# 1. Connection to persistent ChromaDB (legal_corpus & scientific_corpus)
# 2. Validates total chunk record counts and dimensional consistency
# 3. Verifies metadata retention (full text, page, section_id, parent_id)
# 4. Executes sample retrieval queries to confirm vector health
```

Corpus Validation Gate

Never proceed to Stage 11 benchmark evaluation without a clean passing report from **scripts/validate_corpus.py**. If collections show 0 records, verify whether **scripts/build_vector_store.py** encountered empty chunk directories or failed write permissions.

5. Testing & Benchmarking on the New Dataset

Once the original dataset is indexed, run the verification evaluation suite. This tests retrieval precision, source identity filtering, evidence eligibility, and LLM reasoning accuracy against ground truth.

Benchmark Execution Commands

```
# 1. Test hybrid retrieval across exact and semantic modes:
python scripts/test_retrieval.py --domain legal --query "Section 3(d)"

# 2. Test single claim verification end-to-end:
python scripts/verify_claim.py --domain legal --claim "Supreme Court upheld Section 3(d)" --ref "Section 3(
d)"

# 3. Execute Stage 11 Automated Benchmark & Error Analysis Suite:
python scripts/run_benchmark.py --benchmark-file data/evaluation/benchmark.jsonl --output-dir evaluation/
```

5.1 Benchmark Audit Reports

Running `run_benchmark.py` automatically generates two synchronized audit deliverables:

- **evaluation/benchmark_results.json:** Machine-readable evaluation report with overall pass rate, final verdict accuracy, reference resolution accuracy, confusion matrix, stage error counts, and individual case traces.
- **evaluation/benchmark_report.md:** Executive Markdown audit report complete with confusion matrices, divergence summaries, and failure breakdown across the 14 error categories.

5.2 Key Audit Performance Indicators

Audit Metric	Target Threshold	Diagnostic Significance
Overall Pass Rate	≥ **90.0%**	Percentage of cases satisfying verdict, reference status, source availability, and evidence criteria.
Final Verdict Accuracy	≥ **95.0%**	Accuracy of final classification across supported, unsupported, contradicted, not_found.
Reference Resolution Accuracy	**100.0%**	Exact statutory/citation discovery rate vs rejecting invalid references without false semantic matches.
Source Availability Accuracy	**100.0%**	Isolation rate ensuring citing document chunks are never attributed as evidence.

Stage-Level Error Attribution Breakdown

If any case fails in the benchmark, the engine attributes the failure to the earliest diverging stage: **PARSING** → **CHUNKING** → **CLAIM_EXTRACTION** → **REFERENCE_RESOLUTION** → **RETRIEVAL** → **SOURCE_ATTRIBUTION** → **EVIDENCE_SELECTION** → **SEMANTIC_VERIFICATION**. This isolates whether errors stem from data ingestion, retrieval ranking, or LLM reasoning.

6. Troubleshooting & Migration Checklist

6.1 Common Pitfalls & Diagnostic Resolutions

Issue / Symptom	Root Cause	Resolution Procedure
Chroma Lock / Collision	Previous process did not release ChromaDB directory lock.	Terminate all Python background processes. Delete `vector_db/chroma/` before fresh indexing.
CUDA Out of Memory	Batch size too large during embedding computation.	Set `EMBEDDING_DEVICE=cpu` in `.env` or reduce batch size in `scripts/build_embeddings.py`.
Heading Evidence Leak	Structural section heading selected as substantive claim evidence.	Verify chunk's `content_type` is marked `heading`. The verifier automatically rejects headings as independent proof.
Missing Reference Hallucination	Semantic search returns closest vector for nonexistent section.	The exact statutory resolver prevents semantic fallback when reference lookup fails. Ensure `expected_reference_status=not_found`.

6.2 Team Handoff & Migration Verification Checklist

Complete this verification checklist when receiving the original dataset from the partner team:

- [ ] **Step 1: Backup Baseline:** Archive current synthetic fixtures (`data/`) and benchmark results.
- [ ] **Step 2: File Placement:** Deposit real PDF/DOCX files into `data/raw/legal/` and `data/raw/scientific/`.
- [ ] **Step 3: Benchmark Placement:** Replace `data/evaluation/benchmark.jsonl` with the real benchmark suite.
- [ ] **Step 4: Vector DB Reset:** Clear `vector_db/chroma/` to eliminate stale embeddings.
- [ ] **Step 5: Run Ingestion:** Execute `build_corpus` for both legal and scientific domains.
- [ ] **Step 6: Validate Corpus:** Run `python scripts/validate_corpus.py` and verify non-zero record counts.
- [ ] **Step 7: Run Unit Tests:** Run `pytest` to ensure parser and chunker logic remain compatible.
- [ ] **Step 8: Run Stage 11 Benchmark:** Execute `python scripts/run_benchmark.py` and inspect pass rate.
- [ ] **Step 9: Review Audit Reports:** Review `evaluation/benchmark_report.md` and diagnose any stage errors.

> **Repository & Engineering Support**
>
> For schema adjustments or parser rule extensions on complex legal formats (e.g. state amendment acts) or scientific publisher formats, refer to **src.parsing.docling_parser** and **src.chunking.legal_chunker**. All code is under version control at: **github.com/dharani20dharan/Citation-Verification**.